

Practical: Perceptron Learning Law with Decision Regions

Aim

To demonstrate the Perceptron Learning Law with its decision regions using Python and display the output graphically.

Objective

To study the working of perceptron learning law and understand decision boundaries in classification problems.

Important Theory for Practical Writing

Artificial Neural Network (ANN) is a branch of Artificial Intelligence inspired by the human brain. A perceptron is the simplest neural network model used for binary classification problems.

The perceptron receives input values, multiplies them with weights, adds bias, and applies an activation function to produce output.

The perceptron learning rule updates weights continuously until the model correctly classifies data.

The output equation is:

$$y = f(\sum w_i x_i + b)$$

Where:

- x_i = input values
- w_i = weights
- b = bias
- f = activation function

The perceptron separates data using a linear decision boundary.

Short Theory for Journal

- Perceptron is a supervised learning algorithm.
 - It is used for binary classification.
 - It works only for linearly separable data.
 - Weights are updated during training.
 - Decision boundary separates two classes.
-

Algorithm

1. Initialize weights and bias to zero.
2. Provide training data.

3. Calculate net input.
4. Apply activation function.

Code

```
import numpy as np
import matplotlib.pyplot as plt

# training data
X = np.array([[2,3],[3,4],[4,5],[1,1],[2,1],[3,1]])
y = np.array([1,1,1,-1,-1,-1])

# Initialize weights and bias
w = np.zeros(2)
b = 0
learning_rate = 0.1

# Perceptron Learning Algorithm
for epoch in range(20):
    for i in range(len(X)):
        net = np.dot(w, X[i]) + b
        y_pred = 1 if net >= 0 else -1
        w = w + learning_rate * (y[i] - y_pred) * X[i]
        b = b + learning_rate * (y[i] - y_pred)

# Plotting decision regions
x1 = np.linspace(0,5,100)
x2 = -(w[0] * x1 + b) / w[1]

plt.figure()
plt.scatter(X[y == 1][:,0], X[y == 1][:,1])
plt.scatter(X[y == -1][:,0], X[y == -1][:,1])
plt.plot(x1, x2)
```

```
plt.xlabel('x1')
```

```
plt.ylabel('x2')
```

```
plt.title('Perceptron Decision Boundary')
```

```
plt.show()
```